

How event emission actually works

Strip away the mystique and it's just an object holding arrays of functions, keyed by name.

- 1 An emitter keeps one internal object: **event name** → **array of listener functions**. That's the entire data structure.
- 2 Subscribing with **on** doesn't run anything. It just pushes a callback into that name's array.
- 3 Emitting with **emit** looks up that array and calls every function in it, passing along whatever arguments you give.
- 4 Unlike `addEventListener`, none of this touches the DOM. It works on any plain object, anywhere in your app.

This exact pattern is what Node's core modules run on, and what your state library's `subscribe` and `notify` are doing under a different name.

The algorithm, boiled down

Six steps, from the object you initialize to the listener you remove.

<EventEmitter>

01 · INIT

events = {} on the instance

02 · ON(NAME, CB)

Create events[name] array if it's missing

03 · ON(NAME, CB)

Push cb into events[name]

04 · EMIT(NAME, ...ARGS)

Look up the listeners for that name

05 · EMIT(NAME, ...ARGS)

Loop and call each listener with args

06 · OFF(NAME, CB)

Filter cb out of events[name]

One object is the whole data structure

`constructor()``EventEmitter.js`

```
class EventEmitter {  
  constructor() {  
    this.events = {};  
  }  
}
```

- > Keys are event names, values are arrays of callbacks. That's the entire model.
- > Nothing else lives on this class. No DOM references, no external dependencies.
- > Starts empty, so every event name gets created lazily the first time something subscribes.

Subscribing does no work yet

on()

EventEmitter.js

```
on(name, callback) {  
  if (!this.events[name]) {  
    this.events[name] = [];  
  }  
  this.events[name].push(callback);  
}
```

- > The callback is only stored here. It doesn't get called until something emits.
- > The guard clause lazily creates the array only the first time a name is used.
- > Call **on** for the same name twice and you get two listeners, not a replacement.

One loop is the entire publish step

`emit()``EventEmitter.js`

```
emit(name, ...args) {  
  const listeners = this.events[name] || [];  
  listeners.forEach(cb => cb(...args));  
}
```

- > Falls back to an empty array, so emitting an event nobody's listening to is a safe no-op.
- > Every listener runs synchronously, in the order it was registered, with the same arguments.
- > Nothing in this loop depends on the DOM or a browser. It works on any object, anywhere.

Removing a listener needs its reference

`off()``EventEmitter.js`

```
off(name, callback) {  
  if (!this.events[name]) return;  
  this.events[name] = this.events[name]  
    .filter(cb => cb !== callback);  
}
```

- > Guards against unsubscribing from a name that was never registered.
- > Filters the specific callback out **by reference**, so you need the same function you passed to on.
- > That's why anonymous inline functions can't be removed later. Filter has nothing to match against.